

NFT Metadata 高并发设计方案

一、架构设计原则

1.1 核心挑战

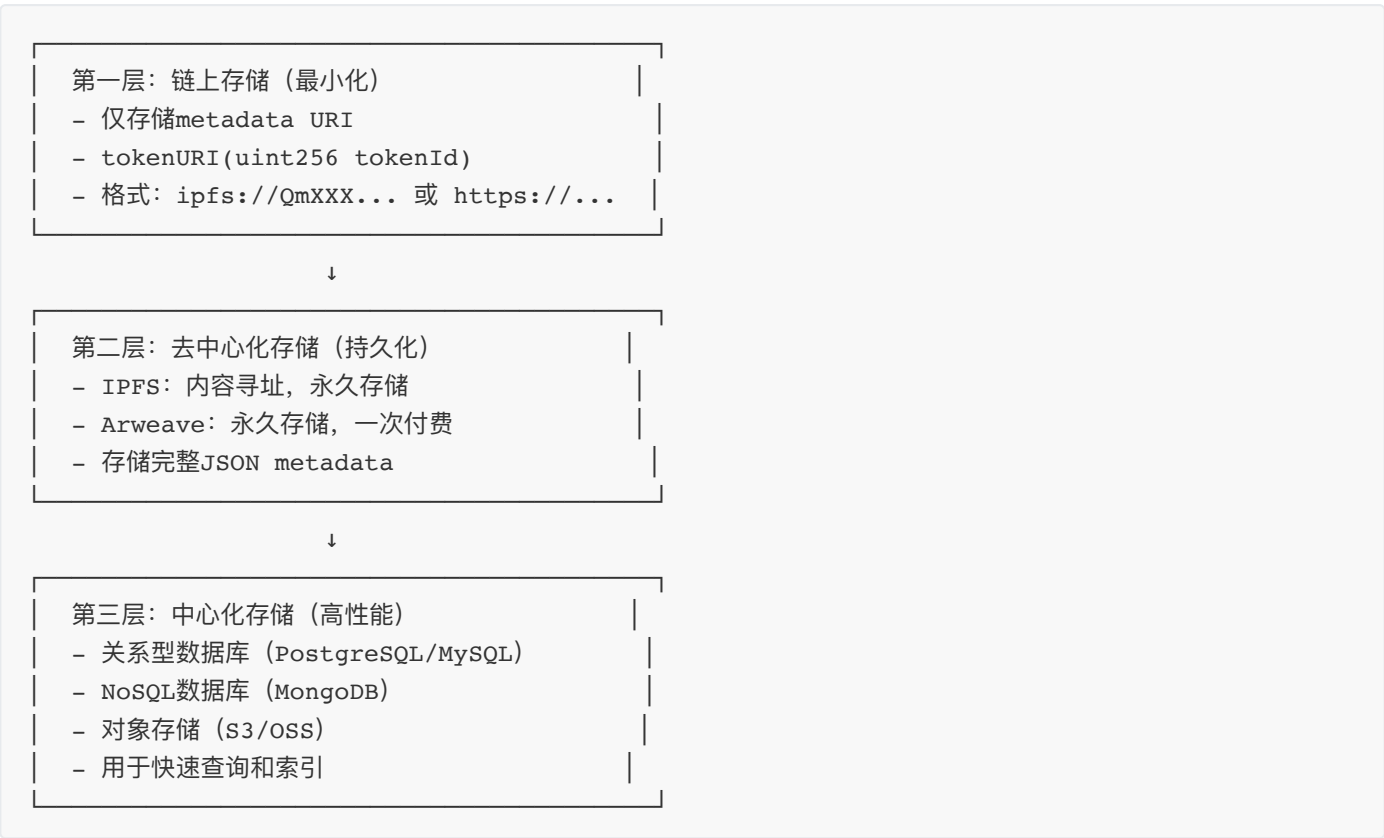
- 高并发读取：NFT市场、钱包、浏览器等场景需要频繁查询metadata
- 数据一致性：链上数据与链下metadata需要保持同步
- 存储成本：大量NFT的metadata存储成本优化
- 访问延迟：全球用户访问需要低延迟响应
- 数据完整性：防止metadata丢失或损坏

1.2 设计原则

- 分层存储：链上存储URI，链下存储完整metadata
- 多级缓存：本地缓存 → Redis → 数据库 → IPFS/Arweave
- 读写分离：读操作走缓存，写操作异步处理
- CDN加速：静态资源通过CDN分发
- 数据分片：按tokenId或collection分片存储

二、存储架构设计

2.1 三层存储架构



2.2 数据库表设计

PostgreSQL 主表设计

```
-- NFT Metadata 主表
CREATE TABLE nft_metadata (
    id BIGSERIAL PRIMARY KEY,
    chain_id INTEGER NOT NULL,
    contract_address VARCHAR(42) NOT NULL,
    token_id NUMERIC(78, 0) NOT NULL, -- 支持uint256
    token_uri TEXT,
    ipfs_hash VARCHAR(64),
    arweave_id VARCHAR(64),

    -- Metadata 核心字段（冗余存储，便于查询）
    name VARCHAR(255),
    description TEXT,
    image_url TEXT,
    animation_url TEXT,
    external_url TEXT,

    -- 属性信息（JSONB存储，支持索引）
    attributes JSONB,

    -- 扩展信息
    collection_id BIGINT,
    creator_address VARCHAR(42),
    owner_address VARCHAR(42),

    -- 状态字段
    status VARCHAR(20) DEFAULT 'active', -- active, deleted, hidden
    is_verified BOOLEAN DEFAULT false,

    -- 时间戳
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    synced_at TIMESTAMP, -- 与链上同步时间

    -- 索引
    UNIQUE(chain_id, contract_address, token_id),
    INDEX idx_collection (collection_id),
    INDEX idx_owner (owner_address),
    INDEX idx_created_at (created_at),
    INDEX idx_status (status)
);

-- 创建GIN索引用于JSONB查询
CREATE INDEX idx_attributes_gin ON nft_metadata USING GIN (attributes);

-- 全文搜索索引
CREATE INDEX idx_name_description_fts ON nft_metadata
```

```
    USING GIN (to_tsvector('english', coalesce(name, '') || ' ' || coalesce(description, '')));
```

MongoDB 设计（可选，用于灵活schema）

```
// MongoDB Collection Schema
{
  _id: ObjectId,
  chainId: Number,
  contractAddress: String,
  tokenId: String, // 使用String避免精度丢失

  // Metadata
  metadata: {
    name: String,
    description: String,
    image: String,
    animation_url: String,
    external_url: String,
    attributes: [
      {
        trait_type: String,
        value: Mixed, // 可以是String或Number
        display_type: String // string, number, date, boost_number等
      }
    ]
  },

  // 存储位置
  storage: {
    ipfs: String,
    arweave: String,
    s3: String
  },

  // 索引字段
  collectionId: ObjectId,
  ownerAddress: String,
  createdAt: Date,
  updatedAt: Date
}

// 索引
db.nft_metadata.createIndex({ chainId: 1, contractAddress: 1, tokenId: 1 }, { unique: true });
db.nft_metadata.createIndex({ "metadata.attributes.trait_type": 1,
"metadata.attributes.value": 1 });
db.nft_metadata.createIndex({ collectionId: 1 });
db.nft_metadata.createIndex({ ownerAddress: 1 });
```

三、缓存架构设计

3.1 多级缓存策略



3.2 Redis缓存设计

缓存Key设计

```
// Key命名规范
const (
    // 单个NFT metadata
    NFTMetadataKey = "nft:meta:%s:%s:%s" // chain:contract:tokenId

    // Collection metadata
    CollectionMetadataKey = "nft:collection:%s:%s" // chain:contract

    // 批量查询
    NFTBatchKey = "nft:batch:%s" // hash(contract:tokenIds)

    // 属性索引
    AttributeIndexKey = "nft:attr:%s:%s:%s" // chain:contract:trait_type:value

    // 热门NFT列表
    HotNFTListKey = "nft:hot:%s" // collection_id
)
```

Redis缓存实现（Go示例）

```
package nft

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

type NFTMetadataCache struct {
    redis    *redis.Client
    local    *sync.Map // 本地缓存
    ttl      time.Duration
}

type Metadata struct {
    ChainID          int           `json:"chain_id"`
    ContractAddress  string        `json:"contract_address"`
    TokenID          string        `json:"token_id"`
    Name             string        `json:"name"`
    Description       string        `json:"description"`
    Image            string        `json:"image"`
    AnimationURL     string        `json:"animation_url"`
    ExternalURL      string        `json:"external_url"`
    Attributes       []Attribute  `json:"attributes"`
    IPFSHash         string        `json:"ipfs_hash"`
    ArweaveID        string        `json:"arweave_id"`
}

// 获取单个NFT metadata (多级缓存)
func (c *NFTMetadataCache) GetMetadata(ctx context.Context, chainID int, contract, tokenID string) (*Metadata, error) {
    // 1. 先查本地缓存
    localKey := fmt.Sprintf("%d:%s:%s", chainID, contract, tokenID)
    if val, ok := c.local.Load(localKey); ok {
        item := val.(*cacheItem)
        if time.Now().Before(item.expireAt) {
            return item.metadata, nil
        }
        c.local.Delete(localKey)
    }

    // 2. 查Redis
    redisKey := fmt.Sprintf("nft:meta:%d:%s:%s", chainID, contract, tokenID)
    val, err := c.redis.Get(ctx, redisKey).Result()
    if err == nil {
        var meta Metadata
        if err := json.Unmarshal([]byte(val), &meta); err == nil {
            // 本地缓存
            item := &cacheItem{
                metadata: meta,
                expireAt: time.Now().Add(c.ttl),
            }
            c.local.Store(localKey, item)
        }
    }
}
```

```

        // 更新本地缓存
        c.updateLocalCache(localKey, &meta)
        return &meta, nil
    }
}

// 3. 查数据库 (这里需要注入数据库客户端)
// meta, err := c.db.GetMetadadata(ctx, chainID, contract, tokenID)
// if err != nil {
//     return nil, err
// }

// 4. 写入缓存
// c.SetMetadadata(ctx, chainID, contract, tokenID, meta)

return nil, fmt.Errorf("metadata not found")
}

// 批量获取metadata (Pipeline优化)
func (c *NFTMetadataCache) BatchGetMetadadata(ctx context.Context, requests
[]MetadataRequest) (map[string]*Metadata, error) {
    // 1. 先批量查本地缓存
    results := make(map[string]*Metadata)
    redisKeys := make([]string, 0)
    redisRequests := make(map[string]MetadataRequest)

    for _, req := range requests {
        key := fmt.Sprintf("%d:%s:%s", req.ChainID, req.Contract, req.TokenID)
        if val, ok := c.local.Load(key); ok {
            item := val.(*cacheItem)
            if time.Now().Before(item.expireAt) {
                results[key] = item.metadata
                continue
            }
        }

        redisKey := fmt.Sprintf("nft:meta:%d:%s:%s", req.ChainID, req.Contract,
req.TokenID)
        redisKeys = append(redisKeys, redisKey)
        redisRequests[redisKey] = req
    }

    // 2. Pipeline批量查Redis
    if len(redisKeys) > 0 {
        pipe := c.redis.Pipeline()
        cmds := make(map[string]*redis.StringCmd)

        for _, key := range redisKeys {
            cmds[key] = pipe.Get(ctx, key)
        }

        if _, err := pipe.Exec(ctx); err == nil {

```

```

        for key, cmd := range cmds {
            val, err := cmd.Result()
            if err == nil {
                var meta Metadata
                if err := json.Unmarshal([]byte(val), &meta); err == nil {
                    req := redisRequests[key]
                    cacheKey := fmt.Sprintf("%d:%s:%s", req.ChainID, req.Contract,
req.TokenID)

                    results[cacheKey] = &meta
                    c.updateLocalCache(cacheKey, &meta)
                }
            }
        }
    }

    // 3. 未命中的查数据库
    // ...

    return results, nil
}

// 设置metadata缓存
func (c *NFTMetadataCache) SetMetadata(ctx context.Context, chainID int, contract, tokenID
string, meta *Metadata) error {
    data, err := json.Marshal(meta)
    if err != nil {
        return err
    }

    redisKey := fmt.Sprintf("nft:meta:%d:%s:%s", chainID, contract, tokenID)

    // 写入Redis
    if err := c.redis.Set(ctx, redisKey, data, c.ttl).Err(); err != nil {
        return err
    }

    // 更新本地缓存
    localKey := fmt.Sprintf("%d:%s:%s", chainID, contract, tokenID)
    c.updateLocalCache(localKey, meta)

    return nil
}

// 更新本地缓存
func (c *NFTMetadataCache) updateLocalCache(key string, meta *Metadata) {
    c.local.Store(key, &cacheItem{
        metadata: meta,
        expireAt: time.Now().Add(5 * time.Minute),
    })
}

```

```

type cacheItem struct {
    metadata *Metadata
    expireAt time.Time
}

type MetadataRequest struct {
    ChainID      int
    Contract     string
    TokenID      string
}

```

四、高并发优化策略

4.1 热点数据优化

问题：热门NFT metadata被频繁访问

解决方案1：本地缓存 + 预热

```

type HotNFTManager struct {
    redis      *redis.Client
    local      *sync.Map
    hotKeys    *sync.Map // 热点key追踪
    threshold  int64     // 热点阈值
}

// 检测热点key
func (m *HotNFTManager) TrackAccess(key string) {
    count, _ := m.hotKeys.LoadOrStore(key, int64(0))
    newCount := count.(int64) + 1
    m.hotKeys.Store(key, newCount)

    // 超过阈值，标记为热点
    if newCount >= m.threshold {
        m.markAsHot(key)
    }
}

// 标记为热点，延长TTL并预热到本地缓存
func (m *HotNFTManager) markAsHot(key string) {
    ctx := context.Background()

    // 1. 延长Redis TTL
    m.redis.Expire(ctx, key, 1*time.Hour)

    // 2. 预热到本地缓存
    val, err := m.redis.Get(ctx, key).Result()
    if err == nil {
        var meta Metadata
        if err := json.Unmarshal([]byte(val), &meta); err == nil {

```



```

        m.local.Store(key, &cacheItem{
            metadata: &meta,
            expireAt: time.Now().Add(30 * time.Minute), // 热点数据本地缓存30分钟
        })
    }
}
}

```

解决方案2：多副本缓存（分散热点）

```

// 为热点key创建多个副本
func (c *NFTMetadataCache) SetHotKeyWithReplicas(ctx context.Context, key string, meta
*Metadata, replicaCount int) error {
    data, _ := json.Marshal(meta)

    pipe := c.redis.Pipeline()
    for i := 0; i < replicaCount; i++ {
        replicaKey := fmt.Sprintf("%s:r%d", key, i)
        pipe.Set(ctx, replicaKey, data, 1*time.Hour)
    }
    _, err := pipe.Exec(ctx)
    return err
}

// 随机访问副本
func (c *NFTMetadataCache) GetHotKeyFromReplica(ctx context.Context, baseKey string,
replicaCount int) (*Metadata, error) {
    replicaIdx := rand.Intn(replicaCount)
    replicaKey := fmt.Sprintf("%s:r%d", baseKey, replicaIdx)

    val, err := c.redis.Get(ctx, replicaKey).Result()
    if err != nil {
        return nil, err
    }

    var meta Metadata
    if err := json.Unmarshal([]byte(val), &meta); err != nil {
        return nil, err
    }

    return &meta, nil
}

```

4.2 批量查询优化

```

// 批量查询优化：合并请求，减少网络往返
type BatchQueryOptimizer struct {
    cache      *NFTMetadataCache
    batchSize  int
    timeout    time.Duration
}

```

```

requests    chan MetadataRequest
results     chan BatchResult
}

func (o *BatchQueryOptimizer) Start() {
    go o.batchProcessor()
}

func (o *BatchQueryOptimizer) batchProcessor() {
    ticker := time.NewTicker(10 * time.Millisecond) // 10ms批处理窗口
    defer ticker.Stop()

    batch := make([]MetadataRequest, 0, o.batchSize)

    for {
        select {
        case req := <-o.requests:
            batch = append(batch, req)
            if len(batch) >= o.batchSize {
                o.processBatch(batch)
                batch = batch[:0]
            }
        case <-ticker.C:
            if len(batch) > 0 {
                o.processBatch(batch)
                batch = batch[:0]
            }
        }
    }
}

func (o *BatchQueryOptimizer) processBatch(batch []MetadataRequest) {
    ctx, cancel := context.WithTimeout(context.Background(), o.timeout)
    defer cancel()

    results, err := o.cache.BatchGetMetadata(ctx, batch)
    if err != nil {
        // 错误处理
        return
    }

    // 发送结果
    o.results <- BatchResult{
        Results: results,
        Error:    err,
    }
}

```

4.3 读写分离

```
// 读操作：优先从缓存读取
```

```

func (s *NFTMetadataService) GetMetadata(ctx context.Context, chainID int, contract,
tokenID string) (*Metadata, error) {
    // 1. 查缓存
    meta, err := s.cache.GetMetadata(ctx, chainID, contract, tokenID)
    if err == nil {
        return meta, nil
    }

    // 2. 查从库 (读从库, 减轻主库压力)
    return s.dbSlave.GetMetadata(ctx, chainID, contract, tokenID)
}

// 写操作: 写主库 + 异步更新缓存
func (s *NFTMetadataService) UpdateMetadata(ctx context.Context, meta *Metadata) error {
    // 1. 写主库
    if err := s.dbMaster.UpdateMetadata(ctx, meta); err != nil {
        return err
    }

    // 2. 异步更新缓存 (不阻塞)
    go func() {
        s.cache.SetMetadata(context.Background(), meta.ChainID, meta.ContractAddress,
meta.TokenID, meta)
    }()

    // 3. 发送消息到消息队列, 异步同步到IPFS
    s.mq.Publish("nft.metadata.updated", meta)

    return nil
}

```

4.4 限流与降级

```

import "golang.org/x/time/rate"

type RateLimitedCache struct {
    cache *NFTMetadataCache
    limiter *rate.Limiter
    fallback func(ctx context.Context, chainID int, contract, tokenID string) (*Metadata,
error)
}

func (r *RateLimitedCache) GetMetadata(ctx context.Context, chainID int, contract, tokenID
string) (*Metadata, error) {
    // 限流保护
    if !r.limiter.Allow() {
        // 降级: 返回默认metadata或错误
        if r.fallback != nil {
            return r.fallback(ctx, chainID, contract, tokenID)
        }
        return nil, fmt.Errorf("rate limit exceeded")
    }
}

```

```
}

return r.cache.GetMetadata(ctx, chainID, contract, tokenID)
}
```

五、数据同步策略

5.1 链上链下同步

```
type MetadataSyncService struct {
    db      *sql.DB
    cache   *NFTMetadataCache
    ipfs    *IPFSClient
    arweave *ArweaveClient
    mq      MessageQueue
}

// 监听链上事件，同步metadata
func (s *MetadataSyncService) SyncFromChain(ctx context.Context, event TransferEvent) error {
    // 1. 从链上获取tokenURI
    tokenURI, err := s.getTokenURIFromChain(event.Contract, event.TokenID)
    if err != nil {
        return err
    }

    // 2. 解析URI，获取metadata
    var metadata *Metadata
    if strings.HasPrefix(tokenURI, "ipfs://") {
        // 从IPFS获取
        metadata, err = s.fetchFromIPFS(tokenURI)
    } else if strings.HasPrefix(tokenURI, "ar://") {
        // 从Arweave获取
        metadata, err = s.fetchFromArweave(tokenURI)
    } else if strings.HasPrefix(tokenURI, "http://") || strings.HasPrefix(tokenURI, "https://") {
        // 从HTTP获取
        metadata, err = s.fetchFromHTTP(tokenURI)
    }

    if err != nil {
        return err
    }

    // 3. 保存到数据库
    if err := s.db.SaveMetadata(ctx, metadata); err != nil {
        return err
    }

    // 4. 更新缓存
```

```

    s.cache.SetMetadata(ctx, metadata.ChainID, metadata.ContractAddress, metadata.TokenID,
metadata)

    // 5. 备份到去中心化存储 (异步)
    go s.backupToDecentralizedStorage(metadata)

    return nil
}

// 异步备份到去中心化存储
func (s *MetadataSyncService) backupToDecentralizedStorage(meta *Metadata) {
    // 1. 上传到IPFS
    ipfsHash, err := s.ipfs.Upload(meta)
    if err == nil {
        meta.IPFHash = ipfsHash
    }

    // 2. 上传到Arweave (可选)
    arweaveID, err := s.arweave.Upload(meta)
    if err == nil {
        meta.ArweaveID = arweaveID
    }

    // 3. 更新数据库
    s.db.UpdateMetadata(context.Background(), meta)
}

```

5.2 消息队列异步处理

```

// 使用Kafka处理metadata更新事件
type MetadataEventProcessor struct {
    consumer  kafka.Consumer
    service    *MetadataSyncService
    workers    int
}

func (p *MetadataEventProcessor) Start() {
    for i := 0; i < p.workers; i++ {
        go p.worker()
    }
}

func (p *MetadataEventProcessor) worker() {
    for {
        msg, err := p.consumer.ReadMessage(context.Background())
        if err != nil {
            continue
        }

        var event MetadataUpdateEvent
        if err := json.Unmarshal(msg.Value, &event); err != nil {

```

```

        continue
    }

    // 处理metadata更新
    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    if err := p.service.SyncFromChain(ctx, event.TransferEvent); err != nil {
        // 错误处理: 重试或记录日志
        log.Errorf("Failed to sync metadata: %v", err)
    }
    cancel()
}
}

```

六、CDN与静态资源优化

6.1 CDN配置

```

// Metadata中的图片URL使用CDN
type Metadata struct {
    Image      string `json:"image"`           // CDN URL:
    https://cdn.example.com/ipfs/QmXXX
    AnimationURL string `json:"animation_url"` // CDN URL
    // ...
}

// CDN URL生成
func GenerateCDNURL(ipfsHash string) string {
    // 方案1: 使用IPFS网关CDN
    return fmt.Sprintf("https://ipfs-gateway.example.com/ipfs/%s", ipfsHash)

    // 方案2: 使用云服务商CDN (已缓存到OSS/S3)
    return fmt.Sprintf("https://cdn.example.com/nft/%s", ipfsHash)
}

```

6.2 图片优化

```

// 图片预处理: 生成多种尺寸
type ImageProcessor struct {
    s3 *s3.Client
}

func (p *ImageProcessor) ProcessImage(originalURL string) error {
    // 1. 下载原图
    img, err := downloadImage(originalURL)
    if err != nil {
        return err
    }

    // 2. 生成多种尺寸

```

```

sizes := []int{100, 300, 600, 1200}
for _, size := range sizes {
    resized := resizeImage(img, size, size)

    // 3. 上传到CDN
    key := fmt.Sprintf("nft/images/%dx%d/%s", size, size, hash(originalURL))
    p.s3.Upload(key, resized)
}

return nil
}

```

七、监控与告警

7.1 关键指标监控

```

type MetadataMetrics struct {
    // 缓存指标
    CacheHitRate    prometheus.HistogramVec // 缓存命中率
    CacheMissRate   prometheus.CounterVec    // 缓存未命中次数

    // 性能指标
    QueryLatency    prometheus.HistogramVec // 查询延迟
    BatchQuerySize  prometheus.HistogramVec // 批量查询大小

    // 错误指标
    ErrorRate       prometheus.CounterVec    // 错误率
    TimeoutRate     prometheus.CounterVec    // 超时率

    // 业务指标
    MetadataCount   prometheus.GaugeVec      // metadata总数
    SyncLag         prometheus.GaugeVec      // 同步延迟
}

// 使用示例
func (s *NFTMetadataService) GetMetadata(ctx context.Context, chainID int, contract,
tokenID string) (*Metadata, error) {
    start := time.Now()
    defer func() {

s.metrics.QueryLatency.WithLabelValues("single").Observe(time.Since(start).Seconds())
    }()

    meta, err := s.cache.GetMetadata(ctx, chainID, contract, tokenID)
    if err != nil {
        s.metrics.CacheMissRate.WithLabelValues("redis").Inc()
        s.metrics.ErrorRate.WithLabelValues("cache_miss").Inc()
    } else {
        s.metrics.CacheHitRate.WithLabelValues("redis").Observe(1)
    }
}

```

```
    return meta, err
}
```

7.2 告警规则

```
# Prometheus告警规则
groups:
- name: nft_metadata
  rules:
    - alert: HighCacheMissRate
      expr: rate(nft_metadata_cache_miss_total[5m]) > 0.5
      for: 5m
      annotations:
        summary: "NFT metadata缓存命中率过低"

    - alert: HighQueryLatency
      expr: histogram_quantile(0.99, nft_metadata_query_latency_seconds) > 1
      for: 5m
      annotations:
        summary: "NFT metadata查询延迟过高"

    - alert: MetadataSyncLag
      expr: nft_metadata_sync_lag_seconds > 300
      for: 10m
      annotations:
        summary: "NFT metadata同步延迟过大"
```

八、最佳实践总结

8.1 存储策略

1. 链上最小化：只存储URI，不存储完整metadata
2. 去中心化备份：重要metadata备份到IPFS/Arweave
3. 中心化加速：使用数据库和CDN提供高性能访问

8.2 缓存策略

1. 多级缓存：本地 → Redis → 数据库
2. 热点优化：识别热点数据，延长TTL，多副本存储
3. 批量优化：合并请求，减少网络往返

8.3 性能优化

1. 读写分离：读从库，写主库
2. 异步处理：写操作异步更新缓存和同步到IPFS
3. 限流降级：保护系统，防止过载

8.4 数据一致性

1. **最终一致性**: 允许短暂的数据不一致
2. **版本控制**: metadata支持版本号, 便于回滚
3. **校验机制**: 定期校验链上链下数据一致性

8.5 扩展性

1. **水平扩展**: Redis集群、数据库分片
2. **垂直扩展**: 增加缓存容量、数据库性能
3. **弹性伸缩**: 根据负载自动扩缩容

九、技术选型建议

9.1 缓存

- **Redis Cluster**: 分布式缓存, 支持高并发
- **Caffeine**: 本地缓存, 超低延迟

9.2 数据库

- **PostgreSQL**: 关系型数据, JSONB支持, 全文搜索
- **MongoDB**: 灵活schema, 适合复杂metadata结构

9.3 去中心化存储

- **IPFS**: 内容寻址, 广泛使用
- **Arweave**: 永久存储, 一次付费

9.4 消息队列

- **Kafka**: 高吞吐量, 适合事件流处理
- **RabbitMQ**: 可靠消息传递

9.5 CDN

- **Cloudflare**: 全球CDN, IPFS网关
- **AWS CloudFront / 阿里云CDN**: 云服务商CDN

十、示例：完整服务实现

```
package nft

import (
    "context"
    "database/sql"
    "time"
```

```

)

type NFTMetadataService struct {
    cache      *NFTMetadataCache
    dbMaster   *sql.DB
    dbSlave    *sql.DB
    ipfs       *IPFSClient
    mq         MessageQueue
    metrics    *MetadataMetrics
}

func NewNFTMetadataService(config Config) *NFTMetadataService {
    return &NFTMetadataService{
        cache:      NewNFTMetadataCache(config.Redis),
        dbMaster:   config.DBMaster,
        dbSlave:    config.DBSlave,
        ipfs:       NewIPFSClient(config.IPFS),
        mq:         config.MessageQueue,
        metrics:    NewMetadataMetrics(),
    }
}

// 获取metadata (主要接口)
func (s *NFTMetadataService) GetMetadata(ctx context.Context, req GetMetadataRequest)
(*Metadata, error) {
    start := time.Now()
    defer func() {

s.metrics.QueryLatency.WithLabelValues("single").Observe(time.Since(start).Seconds())
    }()

    // 1. 多级缓存查询
    meta, err := s.cache.GetMetadata(ctx, req.ChainID, req.Contract, req.TokenID)
    if err == nil {
        s.metrics.CacheHitRate.WithLabelValues("cache").Observe(1)
        return meta, nil
    }

    s.metrics.CacheMissRate.WithLabelValues("cache").Inc()

    // 2. 查数据库 (从库)
    meta, err = s.queryFromDB(ctx, req.ChainID, req.Contract, req.TokenID)
    if err != nil {
        s.metrics.ErrorRate.WithLabelValues("db_query").Inc()
        return nil, err
    }

    // 3. 回写缓存
    s.cache.SetMetadata(ctx, req.ChainID, req.Contract, req.TokenID, meta)

    return meta, nil
}

```

```

// 批量获取metadata
func (s *NFTMetadataService) BatchGetMetadata(ctx context.Context, requests
[]GetMetadataRequest) (map[string]*Metadata, error) {
    start := time.Now()
    defer func() {

s.metrics.QueryLatency.WithLabelValues("batch").Observe(time.Since(start).Seconds())
        s.metrics.BatchQuerySize.Observe(float64(len(requests)))
    }()

    return s.cache.BatchGetMetadata(ctx, convertRequests(requests))
}

// 更新metadata
func (s *NFTMetadataService) UpdateMetadata(ctx context.Context, meta *Metadata) error {
    // 1. 写主库
    if err := s.saveToDB(ctx, meta); err != nil {
        return err
    }

    // 2. 异步更新缓存
    go func() {
        s.cache.SetMetadata(context.Background(), meta.ChainID, meta.ContractAddress,
meta.TokenID, meta)
    }()

    // 3. 发送消息, 异步同步到IPFS
    s.mq.Publish("nft.metadata.updated", meta)

    return nil
}

// 从数据库查询
func (s *NFTMetadataService) queryFromDB(ctx context.Context, chainID int, contract,
tokenID string) (*Metadata, error) {
    query := `
        SELECT chain_id, contract_address, token_id, name, description,
            image_url, animation_url, external_url, attributes,
            ipfs_hash, arweave_id
        FROM nft_metadata
        WHERE chain_id = $1 AND contract_address = $2 AND token_id = $3
        LIMIT 1
    `

    var meta Metadata
    var attributesJSON []byte

    err := s.dbSlave.QueryRowContext(ctx, query, chainID, contract, tokenID).Scan(
        &meta.ChainID, &meta.ContractAddress, &meta.TokenID,
        &meta.Name, &meta.Description, &meta.Image, &meta.AnimationURL,
        &meta.ExternalURL, &attributesJSON, &meta.IPFSHash, &meta.ArweaveID,

```

```

    )

    if err != nil {
        return nil, err
    }

    // 解析attributes
    json.Unmarshal(attributesJSON, &meta.Attributes)

    return &meta, nil
}

// 保存到数据库
func (s *NFTMetadataService) saveToDB(ctx context.Context, meta *Metadata) error {
    attributesJSON, _ := json.Marshal(meta.Attributes)

    query := `
        INSERT INTO nft_metadata
        (chain_id, contract_address, token_id, name, description,
         image_url, animation_url, external_url, attributes, ipfs_hash, arweave_id)
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
        ON CONFLICT (chain_id, contract_address, token_id)
        DO UPDATE SET
            name = EXCLUDED.name,
            description = EXCLUDED.description,
            image_url = EXCLUDED.image_url,
            animation_url = EXCLUDED.animation_url,
            external_url = EXCLUDED.external_url,
            attributes = EXCLUDED.attributes,
            ipfs_hash = EXCLUDED.ipfs_hash,
            arweave_id = EXCLUDED.arweave_id,
            updated_at = CURRENT_TIMESTAMP
    `

    _, err := s.dbMaster.ExecContext(ctx, query,
        meta.ChainID, meta.ContractAddress, meta.TokenID,
        meta.Name, meta.Description, meta.Image, meta.AnimationURL,
        meta.ExternalURL, attributesJSON, meta.IPFSHash, meta.ArweaveID,
    )

    return err
}

```

总结

NFT metadata高并发设计的核心要点：

1. 分层存储：链上URI + 去中心化存储 + 中心化数据库
2. 多级缓存：本地缓存 + Redis + 数据库
3. 读写分离：读从库，写主库，异步更新缓存

4. **热点优化**：识别热点数据，多副本存储，延长TTL
5. **批量优化**：合并请求，Pipeline批量查询
6. **CDN加速**：静态资源通过CDN分发
7. **监控告警**：实时监控关键指标，及时发现问题

通过以上设计，可以支撑百万级NFT的metadata高并发访问。